



Session 14

GitOps & Deployments

ArgoCD · Push vs Pull-based · Blue-Green · Canary · Rolling Update



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

DAY

Monday

TIME

3:00 – 5:30 PM

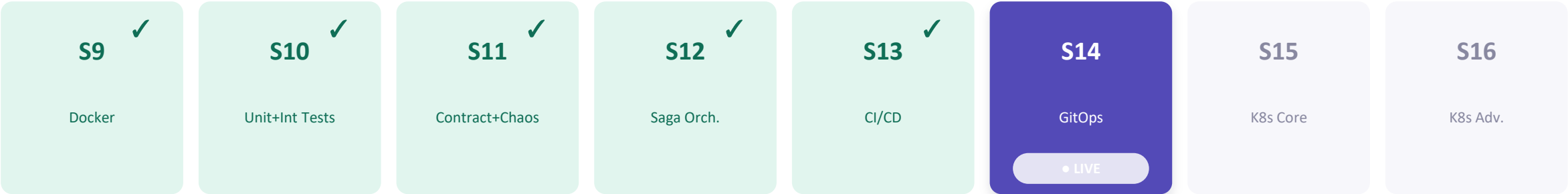
DURATION

2.5 Hours

PHASE

**Phase 2 — Quality &
Deployment**

Where We Are



Session 13 taught the pipeline to build and push. Session 14 teaches the cluster to pull and deploy.

Full 29-session roadmap is in the Kickoff deck only.

Two Halves of the Same Delivery Chain

SESSION 13 — CI (done)

✓ COMPLETE

`Code` → `mvn test` → `Docker build` → `Push image to GHCR`



SESSION 14 — GITOPS (today)

`Registry Image` → `Update Git Manifest` → `ArgoCD Sync` → `Kubernetes Cluster`

Together: every push to main is automatically tested, packaged, and deployed. No manual steps.

Who Audits? Who Detects Drift?

- 1 Our S13 pipeline builds the image, pushes it, then runs `kubectl apply -f k8s/` to deploy. It works.
- 2 Month 2: a production incident. Ops SSHs into the cluster and edits a ConfigMap directly.
- 3 They forget to commit the fix to Git. The fix works — but Git now says something different than the cluster.
- 4 This is DRIFT. Next CI run: `kubectl apply` overwrites their manual fix. The incident returns.
- 5 **Two questions the team cannot answer: who changed the cluster? And when did it drift?**

GitOps solves both: Git is the audit trail. ArgoCD is the drift detector.

Who Applies Changes to the Cluster?

	Push-based CI/CD (S13 only)	Pull-based GitOps (S13+S14)
Who applies changes?	CI pipeline — external, needs cluster credentials	ArgoCD agent — inside cluster, no external credentials
Source of truth?	CI pipeline state (may differ from Git)	Git repository — always
Drift detection?	None — cluster can silently diverge	ArgoCD detects and alerts within seconds
Audit trail?	CI logs, if retained	Git history — every change has author + timestamp
Rollback?	Re-run old pipeline	git revert → ArgoCD auto-syncs previous state
Manual changes?	Silently overridden by next CI run	ArgoCD marks cluster OutOfSync immediately

Push-based CI/CD vs Pull-based GitOps

Simple deploy, small team, one environment?

→ Push-based (simpler)

Multiple environments (dev/staging/prod)?

→ Pull-based GitOps

Need audit trail of who changed what, when?

→ GitOps — every change is a commit

Drift detection: cluster must always match Git?

→ GitOps — ArgoCD syncs constantly

Kubernetes is the deployment target?

→ GitOps is the natural fit

Team prefers no cluster credentials in CI server?

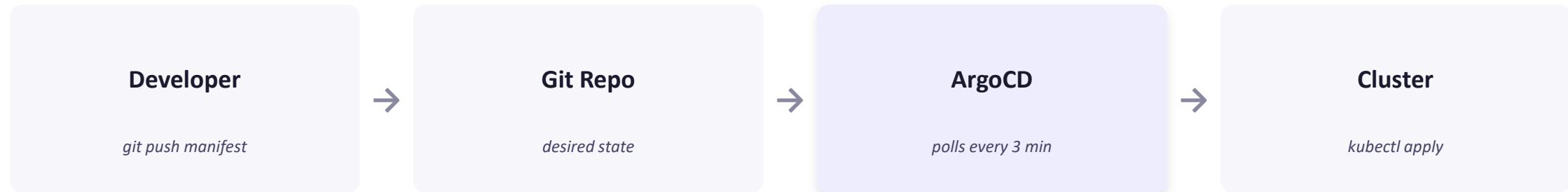
→ GitOps — agent inside cluster pulls

Key insight: GitOps does NOT replace CI. It replaces the deployment step. S13 still tests and builds.

How ArgoCD Works — The Reconciliation Loop



How ArgoCD Works — The Reconciliation Loop



DRIFT SCENARIO

Someone runs `kubectl apply` manually → ArgoCD detects `OutOfSync` → alerts + reverts (if `selfHeal: true`)

If `syncPolicy` is **automated**: ArgoCD applies changes immediately.

If **manual**: shows `OutOfSync`, waits for approval.

ArgoCD Application CRD — Key Fields

Source	Git repo + path + revision to watch	<i>k8s/product-service/</i>
Destination	Cluster + namespace to deploy to	<i>namespace: ecommerce</i>
Sync Policy	automated: deploy on every change · manual: wait for approval	<i>Dev: automated · Prod: manual</i>
Health Status	Healthy / Progressing / Degraded	<i>reflects real pod status</i>
Sync Status	Synced / OutOfSync	<i>reflects Git vs cluster comparison</i>

Kubernetes Manifests: Intro Context Only

Today we cover exactly enough K8s to read a Deployment and connect it to ArgoCD:

Deployment

Service

ConfigMap

This is NOT covered today — explicitly deferred:

- Ingress (external HTTP routing)
- RBAC, Secrets
- HPA (autoscaling)
- StatefulSets, PersistentVolumes

Session 15

Session 15

Session 16

Session 16

“Great question — that is exactly what Sessions 15 and 16 cover.” — validate, name the session, move on.

Kubernetes Deployment — product-service

```
# 1. Deployment — declares desired pod state
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product-service
  namespace: ecommerce
spec:
  replicas: 2                                # run 2 instances
  selector:
    matchLabels: { app: product-service }
  template:
    metadata:
      labels: { app: product-service }
    spec:
      containers:
        - name: product-service
          image: ghcr.io/org/product-service:sha-abc1234
          ports: [{ containerPort: 8081 }]
          resources:
            requests: { cpu: 100m, memory: 256Mi }
            limits:   { cpu: 500m, memory: 512Mi }
```

What to Say When Questions Go Beyond Scope

“How do users access this from outside the cluster?”

→ [Ingress — Session 15](#)

“How do we store passwords securely?”

→ [Kubernetes Secrets — Session 15](#)

“Can Kubernetes scale this automatically?”

→ [HPA — Session 16](#)

“What about stateful databases in Kubernetes?”

→ [StatefulSets + PVs — Session 16](#)

Pattern: validate the question, name the session, move on. Never improvise K8s depth.

Service (ClusterIP) + ConfigMap

```
# 2. Service – internal load balancer (ClusterIP)
apiVersion: v1
kind: Service
metadata: { name: product-service, namespace: ecommerce }
spec:
  selector: { app: product-service }
  ports: [{ port: 8081, targetPort: 8081 }]
  type: ClusterIP          # internal only – not exposed externally
---
# 3. ConfigMap – externalised configuration
apiVersion: v1
kind: ConfigMap
metadata: { name: product-config, namespace: ecommerce }
data:
  datasource.url: jdbc:postgresql://postgres:5432/productdb
  redis.host: redis
  eureka.url: http://eureka-server:8761/eureka/
  # NOTE: passwords never in ConfigMap – use Secrets (S15)
```

Deployment Strategies — Blue-Green / Canary / Rolling

Zero downtime, instant rollback needed?

→ Blue-Green

Test new version with real production traffic?

→ Canary (weight-based)

Simple replacement, small downtime acceptable?

→ Rolling Update (default)

Stateful service with database schema changes?

→ Blue-Green — switch after migration

Limited resources, can't run two full envs?

→ Rolling Update or Canary

Strategy Comparison — Cost, Rollback, Use Case

Strategy	Rollback	Resource Cost	When to Use
Rolling Update	Slow — pods roll back one by one	Low — no extra infra	Default, simple deployments
Blue-Green	Instant — switch back to Blue	High — 2× infra during switch	Zero-downtime, DB migration
Canary	Fast — reduce canary % to 0	Medium — controlled ratio	Risk mitigation, real users

- 🔗 **Platform recommendation:** For the E-Commerce platform:
- Rolling Update for non-critical services (product-service read API).
 - Canary for order-service (new payment flow).
 - Blue-Green for config-server/eureka (infrastructure changes).

Repository structure for GitOps

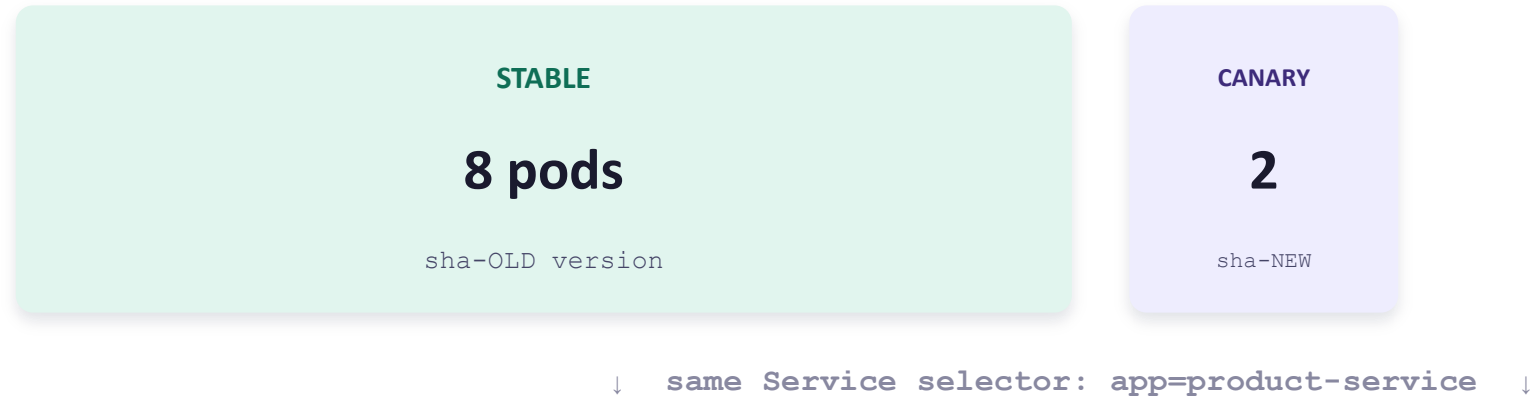
```
# Add this directory to microservices-pro-platform:
microservices-pro-platform/
└─ k8s/
    └─ product-service/
        ├── deployment.yaml    # Deployment manifest (above)
        ├── service.yaml      # Service manifest (above)
        └─ configmap.yaml      # ConfigMap manifest (above)

# ArgoCD watches this path in Git and syncs it to the cluster.
# When CI pushes a new image SHA, update deployment.yaml image field here.
# ArgoCD detects the change → syncs the new image to the cluster automatically.
```

ArgoCD Application CRD

```
# k8s/argocd/product-service-app.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: product-service
  namespace: argocd                # ArgoCD's own namespace
spec:
  project: default
  source:
    repoURL: https://github.com/org/microservices-pro-platform
    targetRevision: main           # watch the main branch
    path: k8s/product-service      # path to manifests in the repo
  destination:
    server: https://kubernetes.default.svc # in-cluster (ArgoCD runs in the same cluster)
    namespace: ecommerce
  syncPolicy:
    automated:                     # AUTOMATIC sync - apply on every Git change
      prune: true                  # delete resources removed from Git
      selfHeal: true               # revert manual kubectl changes (drift correction)
    syncOptions:
      - CreateNamespace=true       # create 'ecommerce' namespace if it doesn't exist
# DEV ONLY: automated sync. Production: remove 'automated:' block and approve manually.
```

Canary via Replica Ratio — 80/20 Split



Kubernetes load-balances by pod count: 8 stable + 2 canary = 10 total → ~80% stable, ~20% canary by probability.

deployment-canary.yaml — 20% New Version

```
# k8s/product-service/deployment-canary.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product-service-canary
  namespace: ecommerce
spec:
  replicas: 2                # 2 out of 10 total = 20%
  selector:
    matchLabels: { app: product-service, track: canary }
  template:
    metadata:
      labels: { app: product-service, track: canary }
    spec:
      containers:
        - name: product-service
          image: ghcr.io/org/product-service:sha-NEW           # new version
# Service selector 'app: product-service' routes to BOTH
# The Service selector 'app: product-service' routes to BOTH stable (8 pods) and canary (2 pods)
# Kubernetes load balances: ~80% to stable, ~20% to canary by pod count
# To promote: increase canary replicas to 10, reduce stable to 0. To rollback: delete canary.
```

ArgoCD UI — Synced + Healthy

`product-service` Synced ✓ Healthy ✓

└─ Deployment: `product-service` (8 pods running)

└─ Deployment: `product-service-canary` (2 pods, track:canary)

└─ Service: `product-service` (ClusterIP, 10 endpoints)

Both stable and canary deployments are visible under one ArgoCD Application — Git remains the single source of truth.

Lab 11B — ArgoCD Application + Canary Deployment

1 Create k8s/product-service/ — Deployment + Service + ConfigMap manifests

2 Create the ArgoCD Application CRD pointing to your Git repo

3 Add deployment-canary.yaml (2 replicas = 20%) and verify traffic split

Acceptance Criteria

- ArgoCD UI shows product-service as Synced + Healthy
- `kubectl get pods` shows at least 2 product-service pods
- 2 pods labelled `track:canary` visible
- selfHeal test: delete a pod → ArgoCD recreates it within 30s

What We Added Today

Capability Added

GitOps Delivery Pipeline

Git is the Source of Truth. Drift detection: cluster always matches Git.

Services Updated

product-service: K8s manifests + ArgoCD Application

Canary deployment: 80% stable / 20% new version

```
Code → (S13 CI) test+build+push → Update Git manifest → (S14 GitOps) ArgoCD syncs → Pods updated
```

Config



Eureka



Gateway



Product



Order



Payment



Inventory



Notif.



product-service is now GitOps-deployed with Canary rollout — the reference pattern for all other services.

5 Errors You Will Hit — And How to Fix Them

ArgoCD Application shows OutOfSync right after creation	Commit k8s manifests to Git BEFORE applying the Application.
Pods stuck in ImagePullBackOff	GHCR private images need an imagePullSecret in the namespace.
Canary pods not receiving traffic	Verify canary selector matchLabels includes app: product-service.
ArgoCD selfHeal not reverting manual changes	Confirm selfHeal: true sits under syncPolicy.automated.
ArgoCD shows Unknown health status	Configure readiness/liveness probes — covered fully in Session 15.

ArgoCD also supports Helm charts as a deployment source — covered in Session 16 (Kubernetes Advanced).

Homework, Checkpoint & What's Next



Daily Quiz — 8 questions, 10 minutes

🔍 Definition of Done — Session 14

- ☐ k8s/product-service/ manifests in Git
- ☐ ArgoCD Application: Synced + Healthy
- ☐ Canary: 2 pods with track:canary label
- ☐ selfHeal verified: deleted pod recreated
- ☐ Commit: session-14: add-k8s-manifests...

🔍 Pre-Session 15 Reading

Topic: Kubernetes Core — Pods, Deployments, Services, Probes

- Pod vs Docker container?
- What does a liveness probe check?
- Why does readiness matter in rolling updates?



SESSION 14 COMPLETE

GitOps Delivery Pipeline Active

Next: Session 15 — Kubernetes Core: Deployments, Services, Ingress, Probes

Wednesday, 3:00–5:30 PM, Online